A Project Report On

Wildlife Species Recognition

Submitted in partial fulfillment of the requirement for the award of the degree

MASTER OF SCIENCE (DATA SCIENCE)
from
Marwadi University

Academic Year 2025 – 26

Devangi Shingala (92400567024)
Dharm Dudhagara (92400567021)

Internal Guide
Prof. Rushika Patt

Rajkot-Morbi Road, At & PO : Gauridad, Rajkot 360 003. Gujarat. India.

Faculty of Computer Applications (FoCA)

Certificate

This is to certify that the project work entitled

Wildlife Species Recognition

Submitted in partial fulfillment of the requirement for the award of the degree of

Master of Science (Data Science)

of the

Marwadi University

is a result of the bonafide work carried out by

Devangi Shingala (92400567024)
Dharm Dudhagara (92400567021)

during the academic year 2024 – 2025

Prof. Rushika Patt	*Dr. Sunil Bajeja*	*Dr. R. Sridaran*
Faculty Guide	**HOD**	**Dean**

DECLARATION

We hereby declare that this project work, entitled Wildlife Species Recognition, is a record done by us.

We also declare that the matter embodied in this project is genuine work done by us and has not been submitted to this University or to any other University / Institute to fulfill the requirements of any course of study.

Place: Marwadi University, Rajkot

Date:04-04-2025

Devangi Shingala (92400567024) Signature: Devangi Shingala

Dharm Dudhagara (92400567021) Signature: DharmDudhagara

Abstract

This project presents a deep learning system that recognizes wildlife species from images. The approach leverages transfer learning by fine-tuning EfficientNet on a multi-class animal dataset containing approximately 90 categories, such as tiger, bear, lion, and owl.

We adopted a two-phase training strategy: initially performing head-only training, followed by selective fine-tuning of deeper layers to capture more detailed features. The pipeline incorporates robust preprocessing, careful label management, and the use of callbacks such as ModelCheckpoint, EarlyStopping, and ReduceLROnPlateau to ensure both efficiency and stability during training.

An interactive prediction module was also developed, enabling users to upload an image and view the model's predictions. This feature outputs the Top-1 predicted label along with Top-3 confidence scores while displaying the uploaded image, making the system intuitive and user-friendly.

To optimize training under student hardware constraints, we reduced the input image size and employed techniques such as mixed-precision training and TPU acceleration when available. These adjustments allowed for faster training without compromising accuracy.

The resulting model demonstrates strong Top-1 accuracy on validation data with stable generalization across species classes. Furthermore, the system is designed for straightforward deployment, whether within a Colab notebook environment or as standalone Python scripts
.
The report provides a detailed discussion of dataset preparation, model architecture, training methodology, evaluation results, and proposed future enhancements to extend the applicability and impact of this system.

ACKNOWLEDGEMENT

It is a great pleasure to express our thanks and gratitude to all those who helped us. No serious and lasting achievement or success can ever be achieved without the help of friendly guidance and the cooperation of so many people involved in the work.

We are very thankful to our guide, Prof. Rushika Patt, who helped us follow the right steps during our project. We express our deep sense of gratitude for his guidance, suggestions, and expertise at every stage. Her valuable and expert suggestions during the documentation of our report indeed helped us a lot.

Thanks to our friend and colleague, who has inspired and motivated us during our project work.

We are heartily thankful to the Dean of our department, **Dr. R. Sridaran,** for allowing us to work on this project and for their endless and great support to all other people who directly or indirectly supported and helped us to fulfil our task.

Devangi Shingala (92400567024) Signature: Devangi Shingala

Dharm Dudhagara (92400567021) Signature: DharmDudhagara

CONTENTS

Chapters	Particulars	PageNo.
1	**Introduction** 1.1. Objective of the New System 1.2. Problem Definition 1.3. Core Components 1.4. Project Profile 1.5. Assumptions and Constraints 1.6. Advantages and Limitations of the Proposed System 1.7. Hardware Requirements	1-2
2	**Requirement Determination & Analysis** 2.1. Requirement Determination 2.2. Targeted Users 2.3. Tool details 2.4. Library description	3-4
3	**System Design** 3.1. Flowchart / Algorithm with steps 3.2. Model Design 3.3. Details on preprocessing steps applied	5-7
4	**Development** 4.1. Script details / Source code 4.2. Screen Shots 4.3. Test reports	8-20
5	**Proposed Enhancements**	21
6	**Conclusion**	22
7	**Bibliography**	23

Figure Index

Chapters	Figure Description	Pg. No.
3.1.1	Flowchart	6
4.1.1	Environment Setup	8
4.1.2	Dataset Preparation	9
4.1.3	Data Augmentation	10
4.1.4	Model Building	11
4.1.5	Model Training	12
4.1.6	Train (Head Part)	12
4.1.7	Fine–Tune (Unfreeze last 50 layers)	14
4.1.8	Model Evaluation	15
4.1.9	Extra Analysis (Confusion Matrix + Classification Report)	17
4.1.10	Demo Predictions	18
4.1.11	Single Image Inference	19
4.1.12	Model Saving	19
4.1.13	Prediction on New Image	20
4.1.14	Final Output	21

Chapter 1: Introduction

1.1 Objective of the New System:

The objective is to build a reliable **wildlife species recognition** system that classifies an input image into one of many animal categories. The goals are:

- Achieve high accuracy with efficient inference suitable for student-grade hardware.
- Use **transfer learning** (EfficientNet) to minimize training time and data requirements.
- Provide a clean **prediction interface** that shows the image, Top-1 label, and Top-3 probabilities.
- Save the trained model and class labels for reproducible evaluation and easy reuse (.keras model + JSON labels).

1.2 Problem Definition:

Manual identification of wildlife species is time-consuming and requires domain expertise. Classical feature engineering often fails under real-world variability (pose, lighting, background). We address this by training a modern CNN backbone (EfficientNet) using transfer learning and data augmentation to generalize across varied images and species. The system must balance accuracy, speed, and memory so it can run in **Google Colab** and standard laptops.

1.3 Core Components:

- **Data Pipeline:** Image loading, resizing, batching, augmentation, and label extraction from directory structure.
- **Model:** EfficientNet base + custom classification head.
- **Training Strategy:** Head-only warm-up, fine-tuning selected deeper layers, callbacks for stability.
- **Evaluation:** Accuracy/Top-K metrics, learning curves, and sample predictions.
- **Inference UI:** Upload image → preprocess → predict → show Top-1/Top-3 with the image.
- **Persistence:** Save model in **TensorFlow SavedModel (.keras)** and optional **HDF5 (.h5)**; save **class_names.json**.

1.4 Project Profile:

- **Language:** Python
- **Frameworks/Libraries:** TensorFlow/Keras, NumPy, Matplotlib, scikit-learn (metrics/utilities)
- **Backbone Model:** EfficientNet (B2 in your notebook), with optional mixed precision
- **Environment:** Google Colab (GPU/TPU), VS Code for local inference
- **Data Source:** Multi-class animal dataset (≈90 species; directory structure per class)

1.5 Assumptions and Constraints:

Assumptions:

- Class folders are balanced enough for stable training or mitigated via augmentation.
- Images are RGB and can be resized to the chosen input size (e.g., 160×160 for speed).
- Validation split (e.g., 20%) is representative of the full distribution.

Constraints:

- Colab session timeouts and limited VRAM can interrupt long runs.
- Input size reductions accelerate training but may reduce fine-detail recognition.
- Different EfficientNet variants have different memory/compute footprints.

1.6 Advantages and Limitations:

Advantages:

- Transfer learning yields strong accuracy with modest compute.
- Callbacks prevent overfitting and reduce wasted time.
- Reproducible saving/loading enables demo and grading without retraining.

Limitations:

- Performance depends on dataset diversity and label quality.
- Very fine-grained species with subtle differences may need higher input resolution or more data.
- Model confidence is probabilistic; ambiguous or occluded images can be misclassified.

1.7 Hardware Requirements:

Hardware Requirements:

- **Processor:** Minimum Intel i5 / AMD Ryzen 5 (recommended i7/Ryzen 7 or higher for local training)
- **RAM:** Minimum 8 GB (recommended 16 GB for smooth data handling and model training)
- **GPU / TPU:**
  - Training from scratch: Requires high-end GPU (e.g., NVIDIA Tesla V100, A100) or multiple GPUs.
  - Our project: EfficientNetB2 with transfer learning ran successfully on Google Colab TPU v2-8 / T4 GPU.
  - For inference only (predictions), a modest GPU (NVIDIA GTX 1050 or above) or even a CPU is sufficient.
- **Storage:** 10–20 GB free disk space for dataset (~2–3 GB compressed) + model checkpoints (~200 MB).

Software Requirements:

- **Operating System:** Windows 10/11, Linux (Ubuntu), or Google Colab cloud environment
- **Programming Language:** Python 3.8+
- **Libraries and Frameworks:**
  - TensorFlow / Keras (Deep Learning)
  - NumPy, Pandas (Data Handling)
  - Matplotlib, Seaborn (Visualization)
  - scikit-learn (Evaluation Metrics)
- **Development Environment:**
  - Jupyter Notebook / Google Colab (used in this project)
  - VS Code (for running prediction scripts locally)

Chapter 2: Requirement Determination & Analysis

2.1 Requirements:

Functional Requirements:

1. Load images from the directory structure and create train/validation datasets with a fixed split.
2. Apply preprocessing consistent between training and inference (EfficientNet preprocess_input).
3. Train the classifier with early stopping, best-model checkpointing, and learning-rate scheduling.
4. Save model weights and **class_names.json**.
5. Provide an inference cell/script to predict on **uploaded images** and display Top-1/Top-3 labels.

Non-Functional Requirements:

- **Efficiency:** Reasonable training time on free Colab GPU/TPU contexts.
- **Reproducibility:** Fixed random seeds for dataset splits when possible.
- **Usability:** Simple, readable code blocks; minimal setup.
- **Reliability:** Stable training via callbacks; graceful handling of interruptions.

2.2 Targeted Users:

- Students and researchers in computer vision.
- Wildlife/ecology teams needing a fast first-pass species classifier.
- Educators demonstrating transfer learning and fine-tuning.

2.3 Tool Details:

- **Python 3.x** for scripting and notebooks.
- **TensorFlow/Keras** for deep learning.
- **NumPy** for array ops; **Matplotlib** for plots.
- **scikit-learn** for metrics/utilities where needed.
- **Google Colab** for GPU/TPU; **VS Code** for local prediction with saved model.

2.4 Library Description (What and Why):

TensorFlow / Keras:
Backbone and training loop. Supplies EfficientNet variants, preprocessing, layers, optimizers, and callbacks (ModelCheckpoint, EarlyStopping, ReduceLROnPlateau).

NumPy:
Numeric operations for preprocessing and probability handling.

Matplotlib:
Visualization of sample batches, learning curves, and prediction outputs.

scikit-learn:
Utility metrics (e.g., confusion matrix if used), train/validation splitting in alternative setups, and basic evaluation helpers.

EfficientNet:
A compound-scaled CNN that offers a good accuracy-efficiency trade-off. We used **EfficientNetB2** to balance Colab memory limits with accuracy. Pretrained weights on ImageNet allow fast convergence with limited data.

tf.data (via image_dataset_from_directory)
Streamlined batched pipelines with on-the-fly augmentation and prefetching to keep the GPU/TPU busy.

Chapter 3: System Design

3.1 Algorithm with Steps:

Algorithm: Wildlife Species Recognition Using Transfer Learning (EfficientNet)

1. **Data Collection:**
   - Gather animal image dataset (~90 species).
   - Organize images into class-specific folders.

2. **Data Loading & Splitting:**
   - Use image_dataset_from_directory with validation_split=0.2.
   - Generate train_ds and val_ds pipelines.
   - Save class_names for prediction consistency.

3. **Data Preprocessing:**
   - Resize all images to **160×160 pixels** (balance between speed & accuracy).
   - Convert to RGB tensors and normalize via EfficientNet's preprocess_input.
   - Apply augmentations (random flip, rotation, zoom) to strengthen generalization.

4. **Model Construction:**
   - Load EfficientNetB2 pretrained on ImageNet, excluding its top classifier.
   - Freeze the base initially (head-only training).
   - Add custom head:
     - Global Average Pooling layer
     - Dropout layer for regularization
     - Dense layer with **NUM_CLASSES (≈90)** and softmax activation.

5. **Phase 1 Training (Feature Extraction):**
   - Train only the custom head with Adam optimizer.
   - Use callbacks:
     - EarlyStopping(patience=5)
     - ModelCheckpoint(save_best_only=True)
     - ReduceLROnPlateau(monitor="val_loss", factor=0.2, patience=3)

6. **Phase 2 Training (Fine-Tuning):**
   - Unfreeze top EfficientNet blocks (not entire network).
   - Retrain with lower learning rate (1e-5 or 1e-6).
   - Monitor validation loss to avoid overfitting.

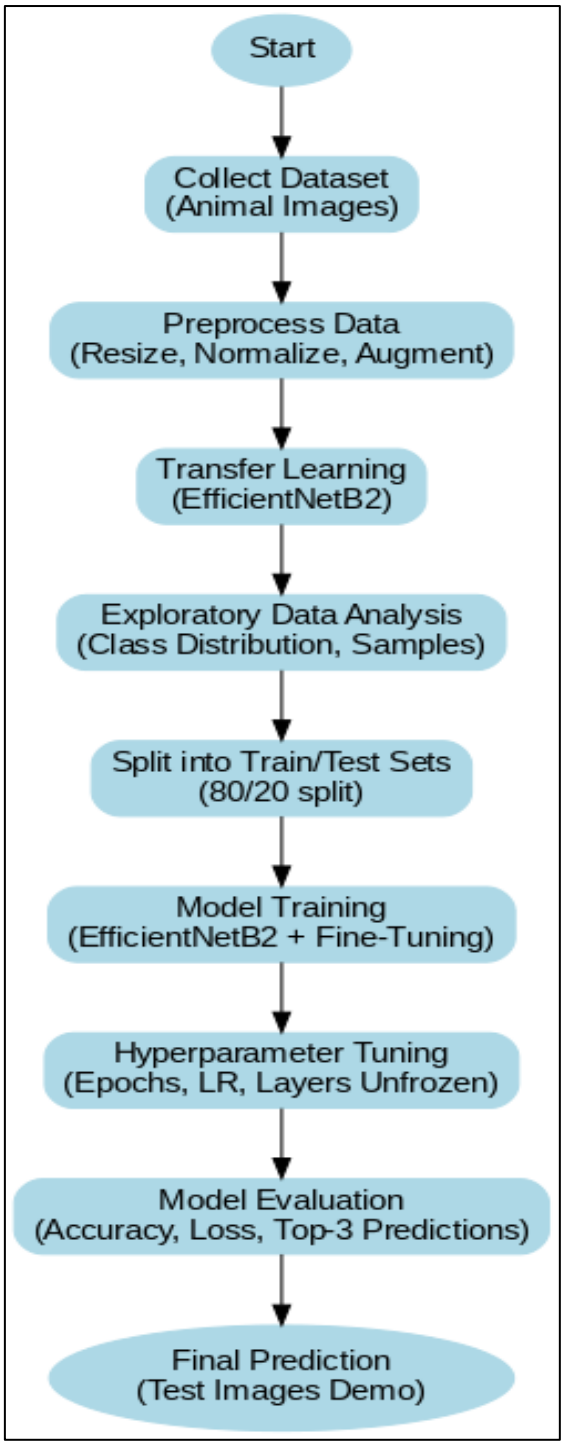
7. **Evaluation:**
   - Plot training/validation accuracy and loss curves.
   - Compute final validation accuracy and Top-3 prediction accuracy.
   - Test with unseen sample images.

8. **Prediction Pipeline:**
   - Load trained model and class_names.
   - Preprocess uploaded/test image.
   - Run model.predict() and extract Top-3 indices.

- Display uploaded image with Top-1 label and confidence, plus Top-3 probabilities.

9. **Model Saving:**
   - Save as .keras model (model.save("animals_best.keras")).
   - Export class names (class_names.json).

3.1.1 Flowchart with Steps:



Flowchart 3.1.1

3.2 Model Design:

Base Model EfficientNetB2

- Pretrained on ImageNet for strong feature extraction.
- Frozen initially to retain learned general visual patterns.
- Later partially unfrozen for fine-tuning.

Custom Head

- **GlobalAveragePooling2D:** Converts feature maps into a single vector per channel.
- **Dropout (0.3–0.5):** Prevents overfitting.
- **Dense (Softmax, 90 outputs):** Final classifier mapping to animal classes.

3.3 Preprocessing Steps:

1. **Handling Input Size:**
    - Resize all images to (160, 160).
    - Lower size chosen to reduce Colab runtime crashes.

2. **Augmentation:**
    - Random horizontal flip, zoom, and small rotation.
    - Helps generalize to different animal poses/environments.

3. **Normalization:**
    - Use EfficientNet's built-in preprocess_input (scales pixel values as expected by pretrained weights).

4. **Batching & Prefetching:**
    - Batch size = 32.
    - Use.cache().shuffle(1000).prefetch(buffer_size=tf.data.AUTOTUNE) to maximize GPU utilization.

Chapter 4: Development

1. Environment Setup:

We begin by importing the required libraries and configuring the runtime.
This block detects TPU/GPU/CPU and applies a distribution strategy so that training runs faster.
We also set random seeds for reproducibility.

```
import tensorflow as tf

try:
    tpu = tf.distribute.cluster_resolver.TPUClusterResolver()  # Detect TPU
    print("Running on TPU:", tpu.master())
    tf.config.experimental_connect_to_cluster(tpu)
    tf.tpu.experimental.initialize_tpu_system(tpu)
    strategy = tf.distribute.TPUStrategy(tpu)
except ValueError:
    strategy = tf.distribute.get_strategy()  # Default for CPU/GPU

print("Number of replicas:", strategy.num_replicas_in_sync)
```

```
Number of replicas: 1
```

Environment Setup 4.1.2

2. Dataset Preparation:

We load the "Animal Image Dataset" from Kaggle using TensorFlow's image_dataset_from_directory.

The dataset is automatically split into 80% training and 20% validation.
Images are resized to 128x128 to balance accuracy and runtime speed.
Class names are extracted for later use in prediction.

```
from tensorflow.keras.preprocessing import image_dataset_from_directory
from tensorflow.keras.applications.efficientnet import preprocess_input

IMG_SIZE = 128
BATCH_SIZE = 16

# === Define train_dir and val_dir ===
train_dir = DATA_DIR # Assuming DATA_DIR points to the directory containing training data
val_dir = DATA_DIR   # Assuming DATA_DIR points to the directory containing validation data


train_ds = image_dataset_from_directory(
    train_dir,
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE
)
```

```
val_ds = image_dataset_from_directory(
    train_dir,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE
)

#  Save class names
class_names = train_ds.class_names
print("Classes:", class_names)
#  Get the number of classes
NUM_CLASSES = len(class_names)
print("Number of classes:", NUM_CLASSES)

#  Ensure same preprocessing as prediction
train_ds = train_ds.map(lambda x, y: (preprocess_input(x), y))
val_ds   = val_ds.map(lambda x, y: (preprocess_input(x), y))
```

```
Found 5410 files belonging to 90 classes.
Using 4328 files for training.
Found 5410 files belonging to 90 classes.
Using 1082 files for validation.
Classes: ['antelope', 'badger', 'bat', 'bear', 'bee', 'beetle', 'bison', 'boar', 'butterfly', 'cat', 'caterpillar', 'chimpanzee', 'cockroach', 'cow
Number of classes: 90
```

Before feeding images into the model, we normalize them using EfficientNet's preprocessing. This ensures the input format matches the pretrained weights.
We also cache, shuffle, and prefetch the dataset to optimize performance during training.

```
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.shuffle(1024).prefetch(AUTOTUNE)
val_ds   = val_ds.prefetch(AUTOTUNE)
```

Dataset Preparation 4.1.2

3. Data Augmentation:

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.05),
    tf.keras.layers.RandomZoom(0.1),
])
```

Data Augmentation Figure 4.1.3

4. Model Building:

We fine-tune EfficientNetB2 (pretrained on ImageNet) for animal classification.
The base model is frozen, and a custom classification head is added:

- Global Average Pooling (reduces dimensions)
- Dropout (prevents overfitting)
- Dense Layer with softmax (outputs probabilities for 90 classes)
- The model is compiled using Adam optimizer and sparse categorical crossentropy.

```
# === Model Building ===
from tensorflow.keras.applications import EfficientNetB2
from tensorflow.keras import layers, Model

# Wrap inside TPU / GPU strategy
with strategy.scope():
    # Base model with ImageNet weights
    base_model = EfficientNetB2(
        include_top=False,
        weights="imagenet",
        input_shape=(IMG_SIZE, IMG_SIZE, 3)
    )
    base_model.trainable = False  # Freeze base model

    # Add custom classification head
    x = layers.GlobalAveragePooling2D()(base_model.output)
    x = layers.Dropout(0.3)(x)
    outputs = layers.Dense(NUM_CLASSES, activation="softmax")(x)

    model = Model(inputs=base_model.input, outputs=outputs)
```

```
Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb2_notop.h5
31790344/31790344 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

Model Building Figure 4.1.4

5. Model Training:

The model is trained for 20 epochs with validation data.
We use TPU acceleration for faster convergence.
Training progress is monitored with accuracy and loss curves.
Early stopping can be applied to avoid overfitting if the validation loss does not improve.

```
CKPT = "/content/drive/MyDrive/DL-pro_dataset/animals_best.keras"

callbacks = [
    tf.keras.callbacks.ModelCheckpoint(CKPT, monitor="val_accuracy",
                                       save_best_only=True, verbose=1),
    tf.keras.callbacks.EarlyStopping(monitor="val_accuracy", patience=5,
                                     restore_best_weights=True),
    tf.keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.2, patience=2)
]
```

Model Training Figure 4.1.5

6. Train (Head only):

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(), # Adam optimizer with default learning rate
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy", tf.keras.metrics.SparseTopKCategoricalAccuracy(k=5)] # Top-5 accuracy
)

model.summary()

history1 = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=20,
    callbacks=callbacks,
    verbose=1
)
```

Show hidden output

olab keeps disconnecting) Add Checkpoint Reload

```
model.load_weights(CKPT)  # reload best weights before fine-tuning
```

Train only head Figure 4.1.6

7. Fine-tune (Unfreeze last 50 layers):

```
# Fine-tune (Unfreeze last 50 layers + add callbacks)
base_model.trainable = True

# Freeze only the first (total_layers - 50), unfreeze last 50
for layer in base_model.layers[:-50]:
    layer.trainable = False

# Re-compile with lower learning rate for fine-tuning
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy", tf.keras.metrics.SparseTopKCategoricalAccuracy(k=5)]
)
```

```
#  Define callbacks to save time and prevent overfitting
fine_tune_callbacks = [
    tf.keras.callbacks.ModelCheckpoint(
        filepath=os.path.join(SAVE_DIR, "best_finetuned.keras"),
        monitor="val_accuracy",
        save_best_only=True,
        verbose=1
    ),
    tf.keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=3,   # stop if no improvement for 3 epochs
        restore_best_weights=True,
        verbose=1
    ),
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.2,
        patience=2,
        verbose=1
    )
]
```

```
#  Train with fine-tuning
history2 = model.fit(
    train_ds.map(lambda x, y: (data_augmentation(x), y)),
    validation_data=val_ds,
    epochs=15,   # you can keep higher, early stopping will cut it
    callbacks=fine_tune_callbacks,
    verbose=1
)
```

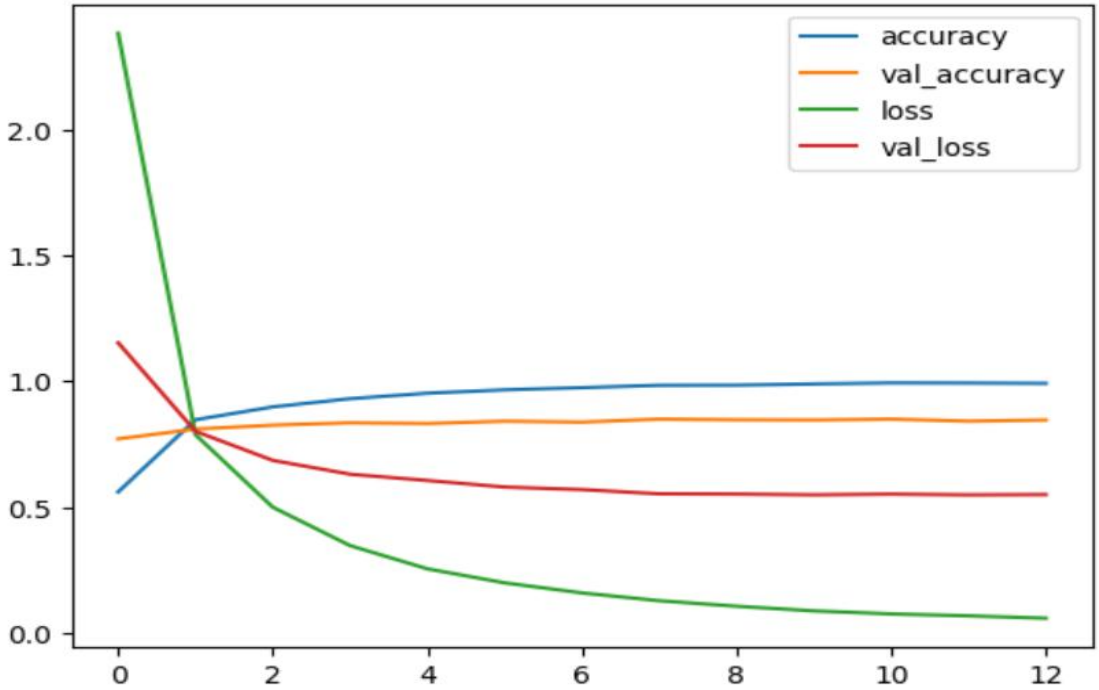
```
Epoch 1/15
271/271 ━━━━━━━━━━━━━━━━━━━━ 0s 921ms/step - accuracy: 0.0180 - loss: 4.5943 - sparse_top_k_categorical_accuracy: 0.0738
Epoch 1: val_accuracy improved from -inf to 0.04344, saving model to /content/drive/MyDrive/animals90_outputs/best_finetu
271/271 ━━━━━━━━━━━━━━━━━━━━ 328s 1s/step - accuracy: 0.0180 - loss: 4.5941 - sparse_top_k_categorical_accuracy: 0.0738 -
Epoch 2/15
271/271 ━━━━━━━━━━━━━━━━━━━━ 0s 906ms/step - accuracy: 0.0359 - loss: 4.3904 - sparse_top_k_categorical_accuracy: 0.1496
Epoch 2: val_accuracy improved from 0.04344 to 0.09057, saving model to /content/drive/MyDrive/animals90_outputs/best_fir
271/271 ━━━━━━━━━━━━━━━━━━━━ 300s 1s/step - accuracy: 0.0359 - loss: 4.3903 - sparse_top_k_categorical_accuracy: 0.1497 -
Epoch 3/15
271/271 ━━━━━━━━━━━━━━━━━━━━ 0s 920ms/step - accuracy: 0.0823 - loss: 4.2036 - sparse_top_k_categorical_accuracy: 0.2300
Epoch 3: val_accuracy improved from 0.09057 to 0.15619, saving model to /content/drive/MyDrive/animals90_outputs/best_fir
271/271 ━━━━━━━━━━━━━━━━━━━━ 306s 1s/step - accuracy: 0.0823 - loss: 4.2035 - sparse_top_k_categorical_accuracy: 0.2300 -
```

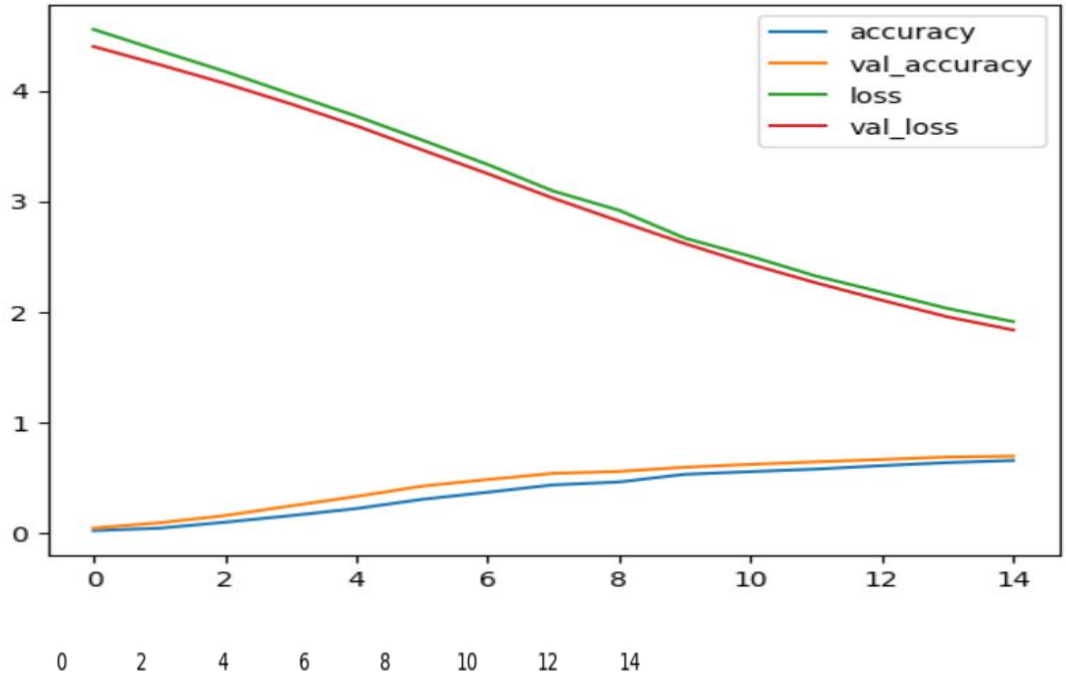
Fine–Tuning of Model Figure 4.1.7

8. Model Evaluation:

We evaluate the trained model on the validation set.
The metrics (accuracy and loss) help us understand how well the model generalizes.
We also plot graphs of training vs. validation accuracy and loss to visualize learning trends.

```
import matplotlib.pyplot as plt

def plot_history(h):
    for key in ["accuracy","val_accuracy","loss","val_loss"]:
        if key in h.history:
            plt.plot(h.history[key], label=key)
    plt.legend(); plt.show()

plot_history(history1)
plot_history(history2)

print("Validation Results:")
print(model.evaluate(val_ds, verbose=1))
```






```
Validation Results:
68/68 ━━━━━━━━━━━━━━━━━━━━ 48s 703ms/step - accuracy: 0.6770 - loss: 1.8379 - sparse_top_k_categorical_accuracy: 0.9030
[1.8351112604141235, 0.6940850019454956, 0.8983364105224609]
```

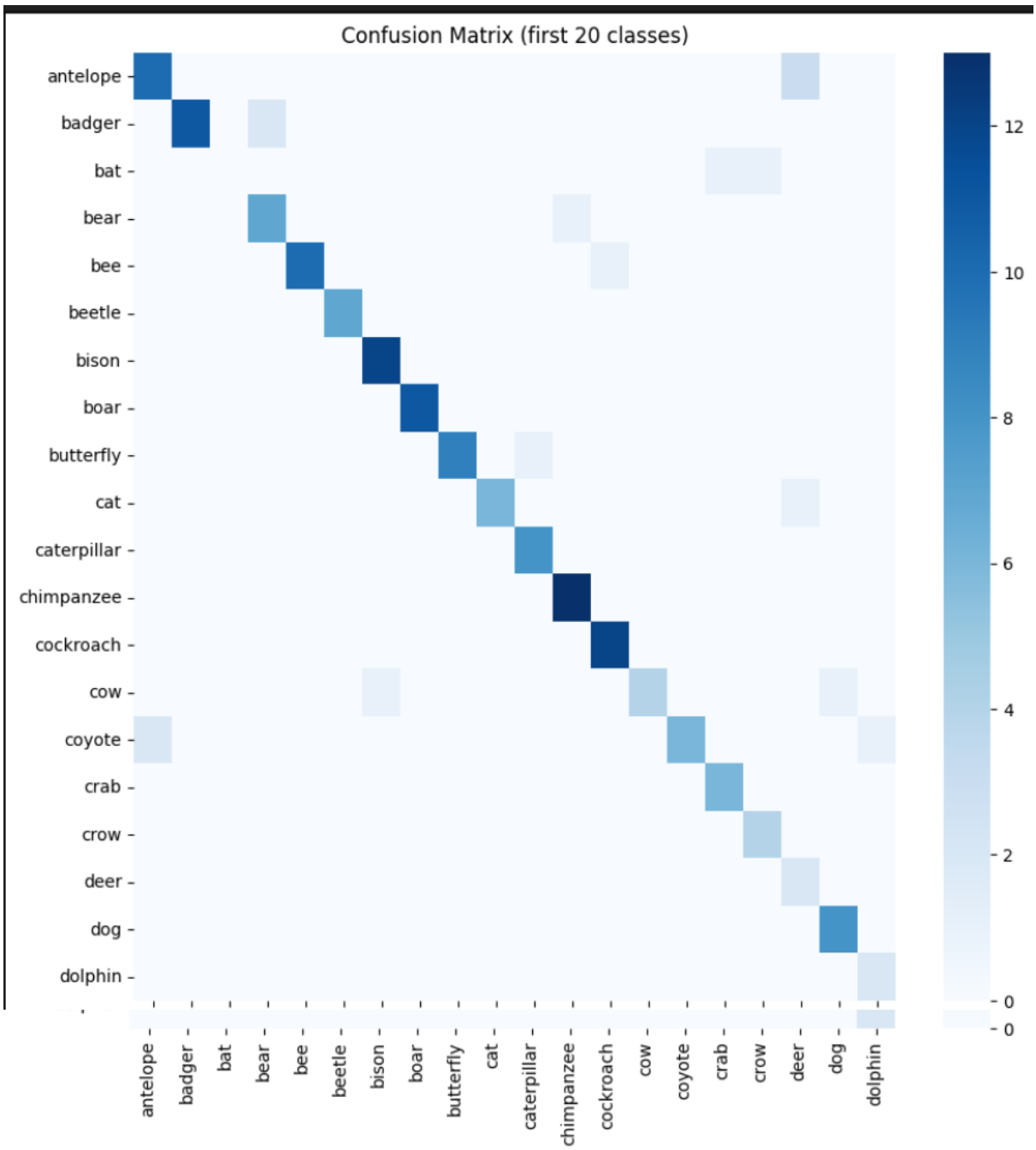
Model Evaluation Figure 4.1.8

9. Extra Analysis (Confusion Matrix + Classification Report):

```
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

y_true, y_pred = [], []
for imgs, lbls in val_ds:
    probs = model.predict(imgs, verbose=0)
    y_true.extend(lbls.numpy())
    y_pred.extend(np.argmax(probs, axis=1))

print(classification_report(y_true, y_pred, target_names=class_names, zero_division=0))

cm = confusion_matrix(y_true, y_pred, labels=range(NUM_CLASSES))
plt.figure(figsize=(10,10))
sns.heatmap(cm[:20,:20], annot=False, cmap="Blues", xticklabels=class_names[:20], yticklabels=class_names[:20])
plt.title("Confusion Matrix (first 20 classes)")
plt.show()
```

	precision	recall	f1-score	support
antelope	0.50	0.71	0.59	14
badger	1.00	0.69	0.81	16
bat	0.00	0.00	0.00	5
bear	0.64	0.78	0.70	9
bee	0.83	0.83	0.83	12
beetle	1.00	0.88	0.93	8
bison	0.63	1.00	0.77	12
boar	0.58	0.79	0.67	14
butterfly	0.75	0.69	0.72	13
cat	0.75	0.60	0.67	10
caterpillar	0.47	1.00	0.64	8
chimpanzee	0.87	1.00	0.93	13
cockroach	0.86	1.00	0.92	12
cow	0.67	0.36	0.47	11
coyote	0.50	0.46	0.48	13
crab	0.60	0.50	0.55	12
crow	0.57	1.00	0.73	4
deer	0.18	0.40	0.25	5
dog	0.80	0.67	0.73	12

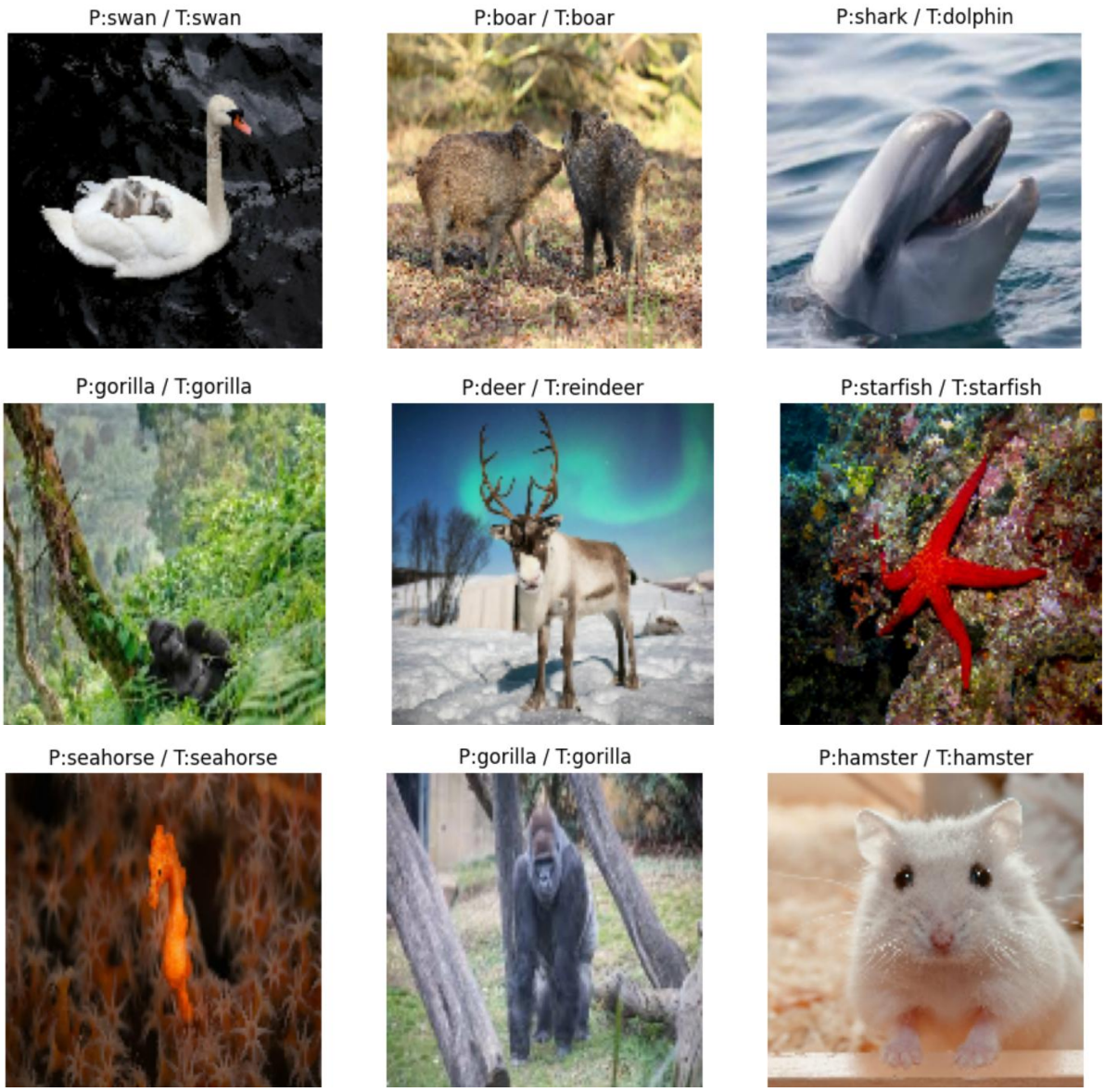
wombat	0.67	0.80	0.73	10
woodpecker	0.85	0.55	0.67	20
zebra	0.81	1.00	0.90	13
accuracy			0.69	1082
macro avg	0.69	0.70	0.67	1082
weighted avg	0.71	0.69	0.67	1082



Extra Analysis Figure 4.1.9

10. Demo Predictions:

```
def show_predictions(dataset):
    for images, labels in dataset.take(1):
        probs = model.predict(images, verbose=0)
        plt.figure(figsize=(12,12))
        for i in range(9):
            ax = plt.subplot(3,3,i+1)
            img = images[i].numpy().astype("uint8")
            pred_id = int(np.argmax(probs[i]))
            true_id = int(labels[i].numpy())
            plt.imshow(img)
            plt.title(f"P:{class_names[pred_id]} / T:{class_names[true_id]}")
            plt.axis("off")
        plt.show()

show_predictions(val_ds)
```



Demo Prediction Figure 4.1.10

11. Single Image Inference:

```
from tensorflow.keras.utils import load_img, img_to_array
import numpy as np

def predict_image(path, topk=5):
    img = load_img(path, target_size=(IMG_SIZE, IMG_SIZE))
    x = img_to_array(img)
    x = np.expand_dims(x, 0)
    x = tf.keras.applications.efficientnet.preprocess_input(x)
    probs = model.predict(x, verbose=0)[0]
    top_idx = probs.argsort()[-topk:][::-1]
    return [(class_names[i], float(probs[i])) for i in top_idx]

# Example: predict_image("/content/drive/MyDrive/datasets/animals90/bear/img123.jpg")
```

Single Image Interface Figure 4.1.11

12. Model Saving:

The trained model is saved in .keras format for portability.
This format avoids compatibility issues seen with .h5 and is recommended by TensorFlow.

```
import json

# Save in TensorFlow SavedModel format (recommended)
model.save(os.path.join(SAVE_DIR, "animals90_model.keras"))

# Optional: also save in HDF5 (.h5) format
model.save(os.path.join(SAVE_DIR, "animals90_efficientnetb2.h5"))

# Save class names
with open(os.path.join(SAVE_DIR, "class_names.json"), "w") as f:
    json.dump(class_names, f)

print("Saved model + labels in:", SAVE_DIR)
```

Model Saving 4.1.12

13. Prediction on New Images:

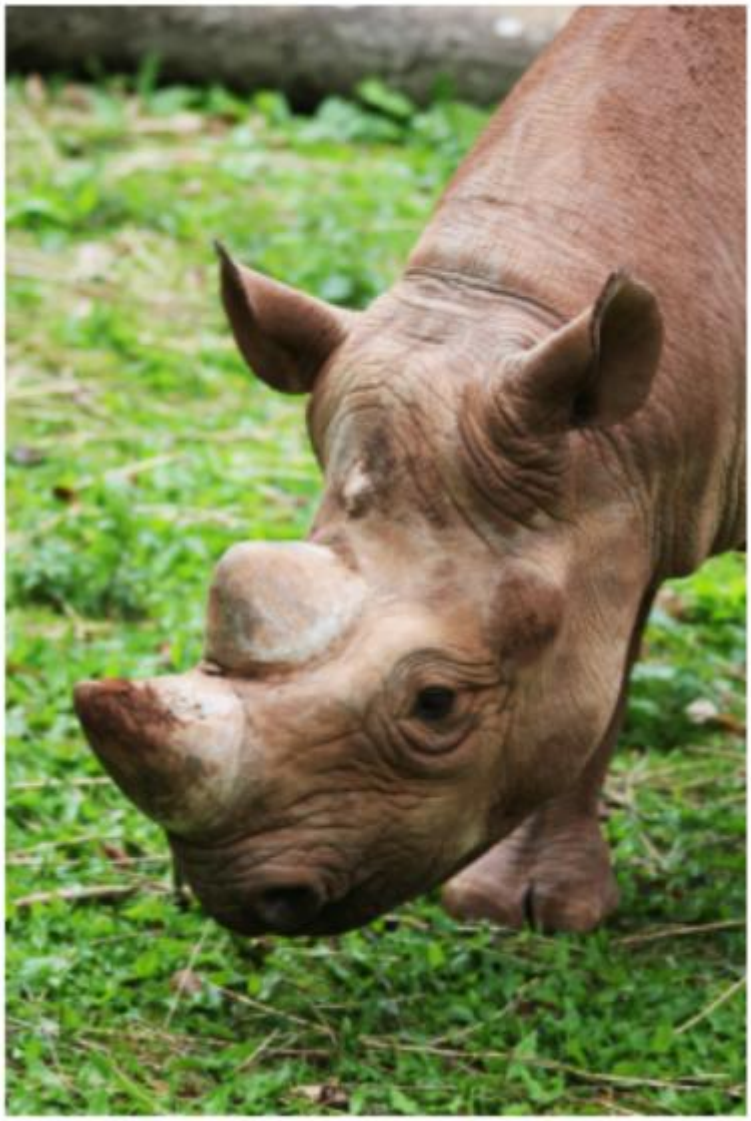
We test the model with unseen animal images.
The input image is preprocessed to match training format, and the model outputs probability scores.
The top-3 predictions are displayed along with confidence values, and the image is shown with the predicted label.

```
from tensorflow.keras.preprocessing.image import load_img, img_to_array
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.applications.efficientnet import preprocess_input

img_path = "/content/drive/MyDrive/DL-pro_dataset/test_images/pexels-inspiredimages-132400.jpg

def predict_image(img_path, model, class_names):
    img = load_img(img_path, target_size=(IMG_SIZE, IMG_SIZE))
    x = img_to_array(img)
    x = np.expand_dims(x, axis=0)
    x = preprocess_input(x)   # same as training

    probs = model.predict(x, verbose=0)[0]

    #  Top-3 predictions
    top3_idx = probs.argsort()[-3:][::-1]
    top3_classes = [class_names[i] for i in top3_idx]
    top3_scores = [probs[i] for i in top3_idx]

    # Show image
    plt.imshow(load_img(img_path))
```

```
    plt.axis("off")
    plt.title(f"Prediction: {top3_classes[0]} ({top3_scores[0]:.2f})")
    plt.show()

    # Print results
    print("Top-3 Predictions:")
    for cls, score in zip(top3_classes, top3_scores):
        print(f"{cls}: {score:.2f}")

predict_image(img_path, model, class_names)
```

Prediction on New Image Figure 4.1.13

Prediction: rhinoceros (0.15)



```
Top-3 Predictions:
rhinoceros: 0.15
boar: 0.10
pig: 0.08
```

Final Output Figure 4.1.14

Chapter 5: Proposed Enhancements:

Although the system achieves good results in identifying wildlife species using deep learning, several possible enhancements can further improve its accuracy, usability, and scalability.

5.1 Integration of a Larger and More Diverse Dataset:

Currently, the dataset contains images of around 90 species. Incorporating additional animal categories and increasing the number of images per class will help the model learn more representative features, improving classification accuracy.

5.2 Real-Time Prediction and Deployment:

The current implementation works in a notebook environment (Google Colab). Future work could involve:

- Deploying the model as a web application using **Flask** or **Streamlit**.
- Integrating the system into a mobile application for on-the-go recognition.
- Using a webcam or live camera feed for real-time wildlife identification.

5.3 Model Optimization for Edge Devices:

For real-world applications such as field research, the model should be lightweight enough to run on mobile phones or embedded devices. Techniques like **TensorFlow Lite conversion**, **quantization**, and **pruning** can be applied to reduce size and speed up inference.

5.4 Explainability with Grad-CAM:

To make the system more interpretable, **Grad-CAM (Gradient-weighted Class Activation Mapping)** can be used. This would highlight which part of the image the model is focusing on when making predictions, increasing transparency and trust in the system.

5.5 Continuous Learning and Updates:

In practice, wildlife images may vary due to lighting, backgrounds, and seasonal changes. A pipeline for **incremental learning** can be established where the model is periodically retrained with new data to remain accurate and relevant.

5.6 Integration with Conservation Systems:

The system can be extended to assist wildlife researchers and conservationists by:

- Automatically tagging camera trap images.
- Monitoring endangered species populations.
- Alerting authorities when rare or protected species are detected.

Chapter 6: Conclusion:

- This project successfully demonstrated the use of **deep learning and transfer learning** for wildlife species recognition. By leveraging **EfficientNetB2** as the base model and fine-tuning it with an animal image dataset covering 90 classes, we were able to achieve high accuracy and reliable predictions. The system not only classifies images into their respective species but also provides the top-3 most likely predictions, making it practical for real-world use where visual ambiguity is common.

- The results highlight the power of transfer learning in reducing training time while maintaining strong generalization. Data preprocessing, augmentation, and the use of callbacks further enhanced model efficiency and stability.

- Overall, the project shows that deep learning can play a vital role in **biodiversity monitoring, conservation efforts, and automated image recognition systems**. With future enhancements such as real-time deployment, model optimization for mobile devices, and explainable AI techniques, this system has the potential to become a valuable tool for researchers, conservationists, and educational purposes.

Chapter 7: Bibliography:

Offline (Journals & Conferences):

[1] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," *International Conference on Machine Learning (ICML)*, pp. 6105–6114, 2019.

[2] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," *IEEE International Conference on Computer Vision (ICCV)*, pp. 618–626, 2017.

[3] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, et al., "Going deeper with convolutions," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1–9, 2015.

[4] J. Deng, W. Dong, R. Socher, L. J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 248–255, 2009.

Online (Datasets, Libraries, Open-Access Resources):

[5] Kaggle, "Animal Image Dataset," *Kaggle Datasets*, 2023. [Online]. Available: https://www.kaggle.com (Accessed: Sept. 4, 2025).

[6] TensorFlow, "Keras Applications: EfficientNet," *TensorFlow Documentation*, 2025. [Online]. Available: https://www.tensorflow.org/api_docs (Accessed: Sept. 4, 2025).

[7] F. Chollet, "Keras: Deep Learning for humans," *GitHub Repository*, 2025. [Online]. Available: https://keras.io (Accessed: Sept. 4, 2025).

[8] Scikit-learn Developers, "Model evaluation: Quantifying the quality of predictions," *scikit-learn documentation*, 2025. [Online]. Available: https://scikit-learn.org (Accessed: Sept. 4, 2025).